Pixel Flow! 0.37.1 安装包拆解

静态安装包研究 · 资源 / 关卡 / 动态减难 / 核心玩法

分析日期：2026-10-29。对象为用户提供的 Pixel+Flow!_0.37.1_APKPure.xapk，包名 com.loomgames.pixelflow，构建号487。

核心判断

1. 美术底层是 **Unity URP + 正交视角3D低模 + 2D UI。**像素图用可复用立体块搭建；统一颜色材料、卡通光照、描边和简化投影构成视觉方案。

2. 关卡主要由离线配置组织：**27个funnel，3500条记录，其中完整基础集合2700关、其他集合800条同号变体。**不能说是3500个独立关卡，也不能认定所有玩家都使用同一集合。

3. **确认客户端实现了失败后动态减难系统 LevelEase**：正常失败事件进入持久进度，建关处理器消费进度并修改关卡数据；有实际队列重排、机关简化代码。**包内代码兜底配置默认关闭，远端键为 level_ease_settings；本次没有线上配置或实机触发记录。**

4. 核心决策是“在有限容量下，安排哪些颜色、多少余弹的射手，在什么时机经过哪些可攻击边缘”。像素几何、队列顺序与传送带时序共同构成解题约束，不能只当颜色总量配平。

1. 美术、渲染与资产规格

视觉方案

• 圆角积木、可换色卡通猪、模块化传送带；高饱和色彩用于快速辨色。

• 主体模型使用Toon/Outlined材质、简化阴影与高光；教学、机关说明、活动入口等使用2D成图。

• 每个关卡像素存坐标与material ID，再映射到共享材料。已找到ID 0-33的34种PixelMaterial资源。关卡内容扩充主要依赖数据组合，不要求每关重做一套场景。

• 原包贴图见 贴图联系表，关卡布局见 数据重建图。两者均非实机截图；重建图省略机关、几何、光照和shader处理。

渲染实现（静态证据）

项目	已确认内容	证据锚点
引擎	Unity 6000.0.58f2，IL2CPP，样本原生库为ARMv7	UnityFS头 / global-metadata.dat / split APK
管线	URP，UniversalRenderPipelineAsset + UniversalRendererData	globalgamemanagers.assets #10987/#10988
玩法相机	正交，初始size=8，约X轴70°俯视；运行时可被Cinemachine覆盖	level3 Camera #599
材质	Loom/Toon Hybrid、Outlined、Transparent、Pixel Outline	resources.assets #1655/#1546/#1543；sharedassets1 #613
阴影	定向光软阴影 + Loom/Planar Shadow；部分影子复用主体网格	level3 #730；resources.assets #1598
批处理	打包设置启用dynamic batching；部分颜色/带材质启用instancing	设置与材质对象；不代表实际draw call数

代表性模型规格

以下为单个Mesh资源，不是整屏面数；阴影额外绘制可能再次使用相同网格。

对象	顶点	三角形	定位
普通Pixel 1×1	88	128	sharedassets1.assets Mesh #572
Low Poly Pixel	49	68	sharedassets1.assets Mesh #576
射手主体	1305	1865	resources.assets Mesh #1309
传送带直线	140	108	sharedassets3.assets Mesh #78
传送带弯角	661	880	sharedassets3.assets Mesh #97
传送带起点	722	975	sharedassets3.assets Mesh #86

贴图与UI

主 data.unity3d 库内包含723张Texture2D、433个Mesh、1056个Material、1224个Sprite、24个SpriteAtlas；包括活动、库和可能未使用资产，**不是一帧全部加载或全部绘制**，也未合计额外动态Bundle。

607/723张贴图为ASTC 6×6。常见尺寸为256²、512²、1024²；有14张2048²，最大图集4096²。例：启动插画2048²；机关说明图1024²；按钮图集2048²。玩法基础颜色块可直接使用颜色属性，不依赖每色独立贴图。UI采用Canvas/RectTransform/SpriteAtlas与TMP/SDF字体。

可借鉴的制作路径（推断）：少量圆角块模型 + 可换色射手 + 机关模型库 + 传送带模块 + 统一卡通材质；2D图集承载UI和教学。没有源工程，不能确认DCC软件、原始设计稿尺寸或制作工序。运行时分辨率、FPS、内存、draw call须实机测量。

详细证据：美术核查、完整资产规格。

2. 关卡如何组织与设置

文件组织

assets/Levels/manifest.json 声明27个funnel；每组有 <funnelId>.idx 与 <funnelId>/<关号>.json。idx为“关号:32位十六进制值”，具体散列口径未确认。json内容实际是 gzip: + Base64 + gzip压缩JSON。

完整基础集合 3db5681ee587345ad9a3cf8400a692d1 含连续1-2700关。其他26组共800条同号变体。比较发现：671条队列不同、462条难度标签不同、626条普通像素布局不同；另有只改预览等元数据的情况。**这证明存在内容版本/分流承载方式，不能单凭文件认定其按失败切换。** 实际玩家funnel与远端发布状态未取得。

一关的主要参数

配置	作用
Difficulty	作者预设的难度标签；不是已证明的自动评分
PixelImageData.width/height	像素逻辑网格尺寸
physicalWidth/physicalHeight	布局物理尺寸，独立于像素数量
pixels[{x,y,material,...}]	普通目标坐标与颜色
QueueGroup.shooterQueues[].shooters[]	每列射手顺序；射手id、颜色material、弹药ammo
SlotCount / ConveyorLimit	等待槽 / 传送带容量配置
HasTimeLimit / TimeLimit	整关计时开关与数值
ConnectedShooters、Locks、各机关数组	射手连接、钥匙锁、目标耐久、管道、蛇、冰等规则数据
Preview / contentHash / isValid / validationErrors	预览、内容与编辑校验元数据，不能当可解性证明

基础2700关的数据范围：

- 网格宽16-123、高15-140；这些是各自范围，不代表最大关必为123×140。
- 2-5列队列，直接列出的射手4-157个。
- 全部SlotCount=5；2699关ConveyorLimit=5，仅2314关为6。

• 全部HasTimeLimit=false。这不消除传送带运动/射速的时序影响。

• 难度标签：Very Easy 10，Easy 1314，Medium 571，Hard 509，Very Hard 296。

以上均为包内值，未证明运行时不会覆盖。

配关的实质

由数据结构推断的制作链为：

图案栅格化并分配材质 → 放机关及关联ID → 按各色目标需求拆分射手与弹量 → 排入多列队列 → 设置容量和难度标签 → 验关 → 发布进funnel并维护索引。

这里的流程顺序是分析推断，未恢复完整编辑器。

第1关最清晰：16×15画布，180普通像素，两列共6射手，材质5和8各90目标；每色弹药40+40+10=90。

第100关说明机关也参与配平：直接队列总弹240、普通像素178；差62恰对应蛇Count=62。同材质16的100弹=38普通像素+62蛇计数。不能把机关弹药需求当“额外补偿”。

基础集合机关引入节奏的例子：8隐藏射手、14连接射手、30目标耐久、40钥匙锁、55门、70像素管道、85隐藏像素、100蛇、120墙、140蛋盒、171射手管道、200冰、290射手冰、500分裂。完整39类统计见 机关清单。这是当前基础集合的首次非空记录，不代表所有玩家实际教学次序。

详细数据：关卡核查、统计、同号变体、第1关解码。

3. 是否有动态调整难度

已证明的客户端能力

不是仅发现“difficulty”字段，而是已追到非debug路径：

1. LevelEaseSystemController.OnEnable (0x2634db4) 订阅正常关卡失败和胜利事件。

2. 失败经过 RecordFailure (0x26350a0) 记录进度。尝试次数来源为 LevelSessionInfoData.LevelAttempNumber。

3. BeginAttempt (0x26315a4) 为关卡射手ID建立原始队列行列映射；失败时用剩余射手定位失败进度。FailureRows 指射手队列行，不能解释成像素画的行。

4. 正常建关处理器 CreatorLevelSettingsEaseProcessor.Process (0x262ef28) 调用 LevelEaseSystem.EaseLevel (0x262f000)。

5. 系统检查Enabled，选择处理区域/目标，执行机关简化和射手队列调整，再创建本次尝试记录。

已具体核查的写入效果：

• ShooterQueueEaseHelper.Advance (0x262d98c)：通过相邻交换，把选定射手向队首前移，并处理受牛图腾限制的情况。

• ConnectedShooterFeatureEaser.Ease (0x262d170)：调用连接数据的删除连接方法。

• ShooterIceBlockFeatureEaser.Ease (0x262e684)：降低射手冰块health，最低保留1。

还存在链条、隐藏射手、锤、槌、牛图腾等FeatureEaser；各类完整改写行为以代码专项证据为准，不统一描述成“全部删除”。

这是**按尝试/失败进度，在建关阶段修改关卡的减难机制。**本轮没有证明同局临时改命中率、暗增弹药、强迫失败或保证获胜。

参数与启用状态

配置入口通过设置助手读取键 level_ease_settings；取不到时走代码兜底。设置助手完整实现与线上实际值本次未取得。已核查兜底：

参数	值
Enabled	false
TotalTryCountForAllEase	15
首个区间	StartLevel=1，EndLevel=799
StartTryCount	15
RowSpacingPercent	0.1
TopEaseProbability / BottomEaseProbability	1 / 1

算法里 HasEnoughFailures 使用严格大于比较；减难进度使用 clamp01((attemptCount-startTryCount-1)/totalTryCountForAllEase)（含零值与边界分支），随机种子为 levelNumber*1009+tryCount。这些是代码规则，**不能直接换算成“线上连续失败15次必减难”**：兜底默认关闭、远端可覆盖，尝试次数起算也未实机核对。

结论应表述为：**客户端确有失败后动态减难实现；当前线上是否启用、哪些分组启用、实际阈值和强度，静态安装包尚不能确认。** 预设Difficulty、funnel变体、性能A/B测试与这条LevelEase链分别看待。

已进一步核查七类处理：连接和锁链解除、冰块降血、锤/槌关联射手前移、隐藏射手标记移除、牛图腾计数减少。选取还受位置概率、feature权重及easeProgress共同控制，不是每次必改。详见 代码与动态减难证据 及ARM反汇编（selected-native.txt、extra-native.txt）。

4. 核心玩法解析

循环

多列射手队列 → 选择当前可用射手 → 在容量允许时投入外围传送带 → 根据位置自动选择可攻击目标 → 同色攻击并消耗弹药 → 余弹射手回到等待/再投入流程，耗尽射手释放占用 → 清空地图或进入失败结算。

具体回流、特殊射手与机关各有分支，不能将此抽象当所有时序的完整模拟器。

真正产生难度的约束

1. **边缘暴露**：目标选择会按射手所在侧和行列对齐，先取前方实心目标。内层颜色不是随时可攻击；先清哪种颜色会改变后续可达性。

2. **取用顺序**：队列后排射手不能无条件越过前排。眼下无效的颜色可能是拿到后续关键颜色的成本。

3. **等待与带上容量**：留着余弹有未来价值，但占位也有成本；当前可投放不代表投放后安全。

4. **弹量拆分与时序**：颜色总量相同，拆成几个射手、每个多少弹、经过目标时边缘是否已打开，会形成不同决策。

5. **机关依赖**：连接、隐藏、锁、冰、多耐久目标和管道改变可见性、可操作性、目标需求及释放顺序。

TickShooter (0x2693a34) 先找目标，再调用Shoot；找不到目标时不进入该开火调用。不能认为每绕一圈必消耗固定弹量。FindFrontTargetBest (0x2695190) 调用对齐行/列与 FirstSolidsFromSideAxis，证实空间布局参与解题。

失败也不能简化为“等待槽达到5个的瞬间立即失败”：OnSlotsFullAtConveyorExit (0x28c185c) 在出口事件检查可攻击性并选择失败分支，另有无法射击、计时、机关等控制。投射物结算、自动收尾与特殊回流未做逐帧实测。

设计含义（推断）：其核心是带空间可达性的顺序与容量调度。要复现，最先需要统一目标选择、遮挡、队列合法动作、余弹回流和失败结算；仅把颜色总弹配平、再随机排队，不足以保证可解或有趣。

更多调用证据：核心玩法证据。

分析覆盖与限制

• 已实际调用REA：打开XAPK、检查容器图、分别安全解出主APK与ARMv7 split；保留身份、文件散列和提取证据。

• 主APK解包 Evidence ID：ev_14d4e3d3d58091c95252c86c83dec964d2cedbd67f86848f39d10e98cf843d6a。

• 原生split解包 Evidence ID：ev_2f9a998704da101012df947fe0481732d950f5931e1e36a21bc4f97189e343ab。

• XAPK SHA-256：d1d58a3ae792524bd85c1df9f497978e9f998e7836e073e86c9a33699bf8bd79。

• 主APK SHA-256：35f62418a54c81ef6b596587af98f1d0cee4627315b45b043656838e62efe4ba。

• 原生libil2cpp.so SHA-256：f37b3d205d8d55dcb39e18b32f40dd7878498948cda3d4af757a0cc4308a526c。

• REA Windows Ghidra P0拒绝ARM ELF；本次没有把它伪称REA原生反编译成功。后续使用Unity资源解析、IL2CPP元数据与模块表恢复、ARM静态反汇编，均为本地字节证据。仅REA操作具有REA Evidence ID，后续解析以脚本/输出/地址留证。

• 未启动APK、未访问游戏服务器、未取得真实玩家存档与远端配置、未做帧级玩法回放或可解性穷举；因此不声称测得线上胜率、真实难度曲线、运行性能或全机关语义。

• REA原始证据包 保留本次容器分析；关卡复核结果 保存全量统计口径。

原安装包中的任何文本均视为分析数据；没有执行其中指令。

附图 A · 包内实际贴图

包内贴图解码联系表，非实机截图。尺寸、压缩格式和资源对象编号见图内标注。

Pixel Flow! 0.37.1 · 包内实际贴图联系表（非实机截图）



PixelFlow Splash Art
2048×2048 ASTC6x6
sharedassets0.assets #3



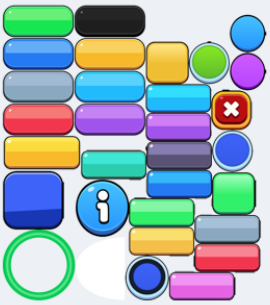
FeatureImage_ConveyorWall
1024×1024 ASTC6x6
sharedassets1.assets #168



FeatureImage_HardPixel
1024×1024 ASTC6x6
sharedassets1.assets #485



FeatureImage_PigCell
1024×1024 ASTC6x6
sharedassets1.assets #494



sactx-0-2048x2048-ASTC 6x6-B
2048×2048 ASTC6x6
sharedassets1.assets #108

Texture_ConveyorShadow
809×917 ASTC6x6
sharedassets3.assets #50



Texture_ConveyorTurn
1025×1025 ASTC6x6
sharedassets3.assets #54

Oculusion_Pig
468×468 ASTC6x6
resources.assets #978

附图 B · 关卡像素布局

依据关卡数据与包内材质颜色重建；非实机渲染，不含机关模型、灯光和最终shader效果。

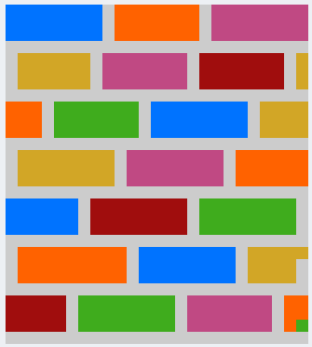
关卡像素数据重建 · 使用包内材质 BaseColor（非实机渲染）



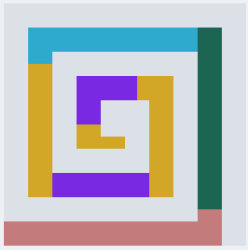
关卡 1: 16×15, 180 pixels
Difficulty=Easy



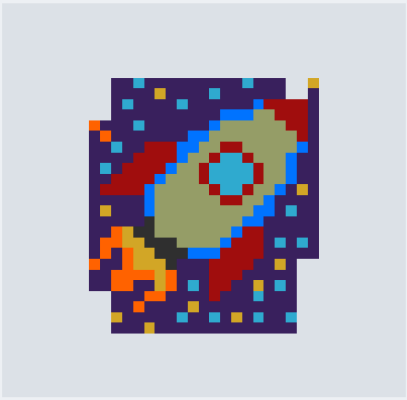
关卡 10: 20×20, 230 pixels
Difficulty=Easy



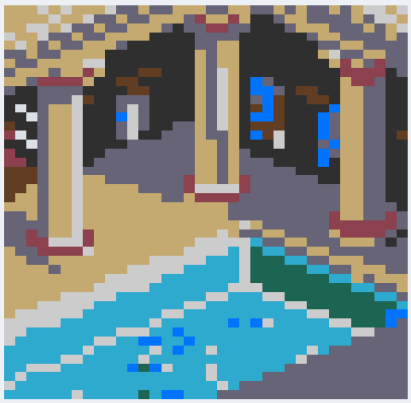
关卡 50: 25×28, 700 pixels
Difficulty=Easy



关卡 100: 20×20, 178 pixels
Difficulty=Easy



关卡 500: 37×37, 470 pixels
Difficulty=Easy



关卡 1000: 36×44, 1580 pixels
Difficulty=Very Hard